23CSC04 - Principles of Compiler Design.

Source Pgm → Compiler → Machine code

↕

Compilation error.

Language Processing System.

Steps for executing a Program:-

Source Pgm (HLL) → stdio.h → conio.h → string.h

Printf(), clrscr(), Strlen()

↓

Preprocessor ← removes all # include directives & all # define directives & includes all file inclusion & with macro expansion. & also all comments.

↓ Modified source Pgm (Compiling code with no preprocessor stmnts). (Pure HLL)

Compiler (Translator)

↓ Assembly lang. (Mnemonic codes)

Assembler (Translator)

↓ Relocatable M/c code.

Linker/Loader

↓

Machine code.

1. Logical address
2. Physical address.
3. offset.

Logical address
→ Pgm generated address.

Logical address + offset = Phy. addre

Physical address
→ Address available in main m/y.

Linker → It links the relocatable m/c code of various library files to main pgm & combines them to produce an executable file (.exe)

Loader → loads .exe file into RAM for execution & then CPU executes our program.

Structure of compiler:-

→ 2 Parts (1) Analysis (2) Synthesis.

⇓ determines the ~~implies~~ operations implied by the source Pgm which are recorded in a Tree Structure (Front end)

(Lexical Analyzer, Syntax Analyzer, Semantic Analyzer)

⇓ Takes the tree Structure & translates the operations therein into the target Pgm (Back end).

(Intermediate code generator, Code Optimizer, Code generator)

Phases of a Compiler

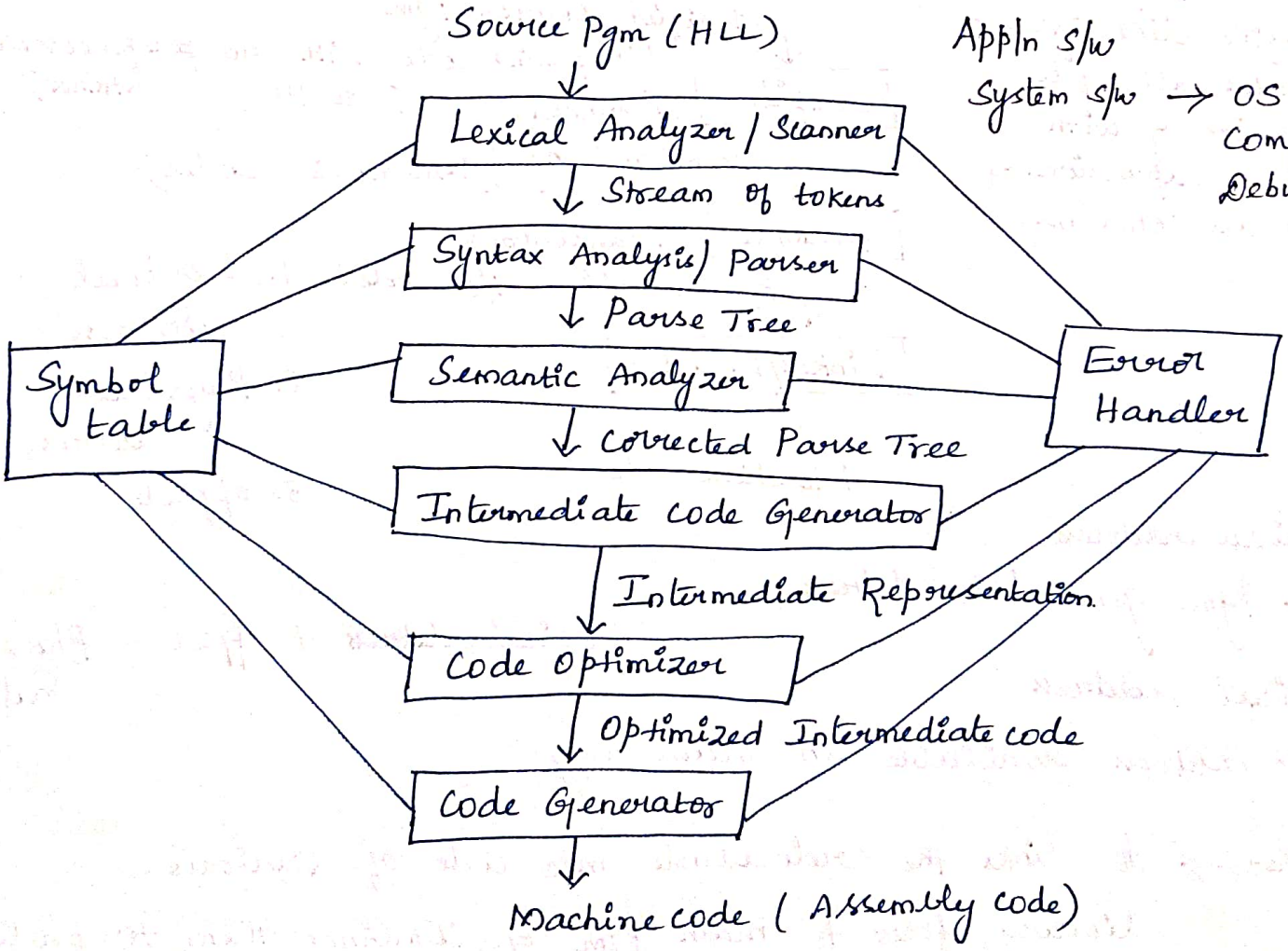
Compiler is a system software

Appln s/w

System s/w → OS

Compiler

Debugger



Phases of a compiler

1. Lexical analyzer reads the source program character by character & converts them into a meaningful sequences called Lexemes.
   ↳ A Meaningful seq. of characters.

   a = b+c*60;

→ Each Lexemes is represented in the form of tokens.

→ Token may be Keyword / identifier / operator / constant.

Each token is represented as, <tokenname, attribute value>.
(used to identify the token easily in the symbol table)
Operator, constants will not have attribute value.

Lexemes	Token
a	identifier
=	operator
b	identifier
+	operator
c	identifier
*	operator
60	constant.

→ Tokens are maintained (stored) in a symbol table which is a Data structure contains about tokens.

→ Each token is represented

<id, 1>, <=>, <id, 2>, <+>, <id, 3>, <*>, <60>

→ a = b+c*60
id1 = id2 + id3 * 60

2. <u>Syntax Analysis</u> / Parser.

→ checks for the syntax of this source pgm is correct / not.

→ If syntax is not correct, error handler will report the errors to the users.

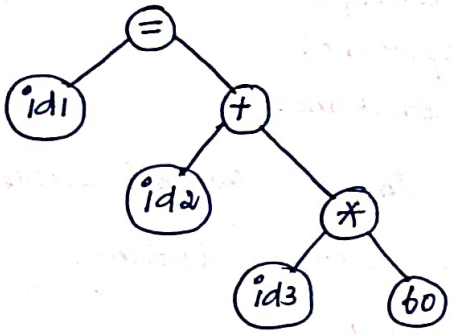
→ If syntax is correct, it generates Parse tree.

→ For constructing Parse tree, it uses operator as internal nodes (Parent node) and left side & right side will be identifier / constants / keywords.

→ While generating Parse tree, highest priority operator is evaluated first, then with the lowest priority operator.

→ $\underbrace{id1}_{LHS} = \underbrace{id2 + id3 * 60}_{(assigned)\ RHS}$ =, +, *

→ So, RHS exp. should be evaluated first.



Parse tree.

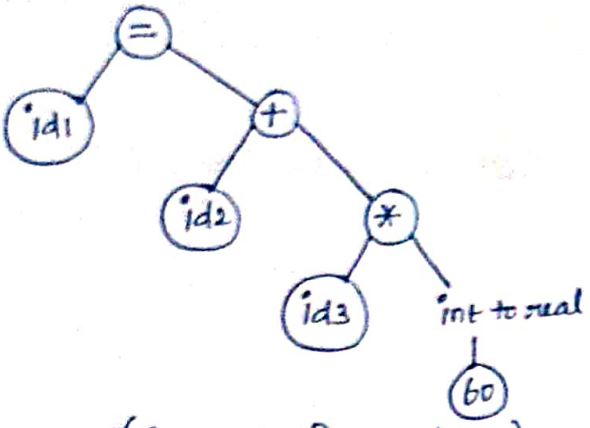
3. <u>Semantic Analysis</u>

→ checks whether meaning of this Parse tree is correct / not.

→ For this purpose, it uses a tool called <u>type checker.</u>

→ It checks for the correctness of data type & if any type conversion is needed, it will perform it.

→ if any variable data type is float, for ex; let b, c → float
60 → int.
float * int → float
we want to store float value then 60 must be converted to 60.0



(Correct Parse tree)

→ it checks whether Parse tree meaning is correct / not.

4. Intermediate code Generation.

→ can be represented using Three address code / Polish notation.

→ Here, TAC is used for representation. (Instructions should contain 3 addresses).

→ 3 Points in TAC,

1. A TAC assignment ins. must have atmost one operator on the RHS. = operator.
2. A compiler must generate a temporary variable for storing the result.
3. Some ins. will contain fewer than 3 ~~addresses~~ operands.

t_1 = int to real (60)

t_2 = id3 * t_1

t_3 = id2 + t_2

id1 = t_3

5. <u>Code Optimization</u>

→ Code Optimizer.

→ In a pgm, some instructions are not needed & can also be removed / combined so that, the length of the pgm can be reduced.

→ $t_1 = id3 * 60.0$
$id1 = id2 + t1$ } 2 line code optimized.

6. <u>Code generation</u>

→ Code generator. → produces (Machine code)

→ uses Assembly language code. words as,

Load – LD Reg, M
Store – ST M, Reg
ADD – R1, R2 / R1, R2, R3
MUL –

→ LDF R2, id3
MULF R2, R2, #60.0 (# used to indicate constant).
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1

(Machine code / Assembly code).

Ex1:- Consider c code:

```
int i, j;
i = i * 70 + j + 2;
```

Write the O/P of all phases of compiler.

Symbol table

1	id	....
2	id	....

$i = i * 70 + j + 2$

↓

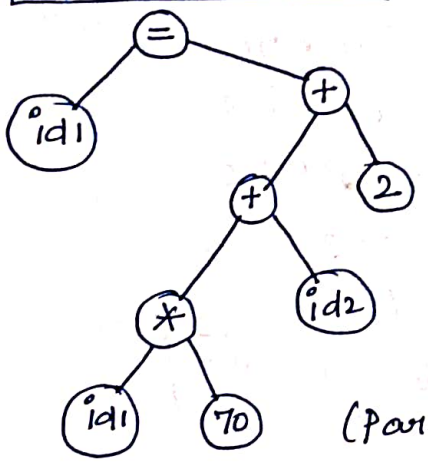
Lexical Analyser

↓

$id1 = id1 * 70 + id2 + 2$ (Stream of tokens)

↓

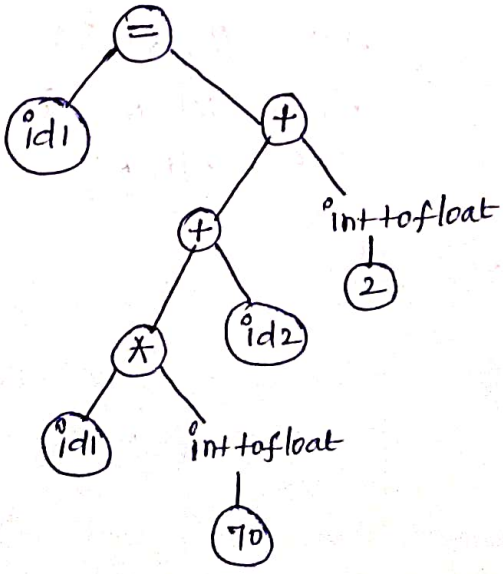
Syntax Analyser



(Parse tree)

↓

Semantic Analyser



↓

Intermediate code Generator

↓

```
t1 = inttofloat(70)
t2 = id1 * t1
t3 = t2 + id2
t4 = ~~t3+~~ inttofloat(2)
t5 = t3 + t4
id1 = t5
```

↓

Code optimizer

↓

```
t1 = id1 * 70.0
t2 = t1 + id2
t3 = t2 + 2.0
id1 = t3
```

↓

Code generator

↓

```
LDF   R1, id1
MULF  R1, R1, #70.0
LDF   R2, id2
ADDF  R1, R1, R2
ADDF  R1, R1, #2.0
STF   id1, R1
```

$x = (a+b) * (c*d)$

↓

Lexical Analyzer

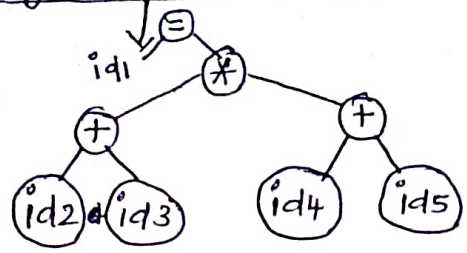
↓

$id1 = (id2 + id3) * (id4 + id5)$

↓

Syntax Analyzer

↓

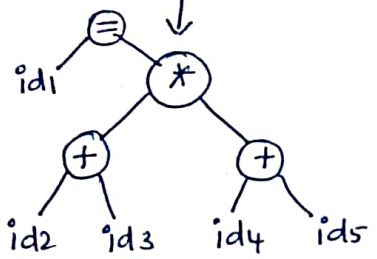


Parse Tree

↓

Semantic Analyzer

↓



↓

Intermediate code generator

↓

$t1 = id2 + id3$

$t2 = id4 + id5$

$t3 = t1 * t2$

$id1 = t3.$

↓

Code Optimizer

↓

$t1 = id2 + id3$

$t2 = id4 + id5$

$id1 = t1 * t2$

↓

Code Generator

```
LD R1, id2
ADD R1, R1, id3
LD R2, id4
ADD R2, R2, id5
MUL R1, R1, R2
ST id1, R1.
```